ارسال یک درخواست تا تولید اولین پاسخ سپری می‌شود. این معیار، زمان پاسخ نامیده می‌شود که زمان سپری شده‌ای است تا اولین پاسخ شروع شود، نه زمانی که برای نمایش پاسخ گرفته می‌شود.

مطلوب‌تر است که بهره‌وری پردازنده و بازده بیشینه شود و زمان گردش، انتظار و پاسخ به حداقل میزان خود برسد. در بیشتر موارد، اندازه‌ی متوسط را بهینه می‌کنیم. با این وجود، در بعضی شرایط علاقه داریم که به جای میانگین، مقادیر حداقل یا حداکثر بهینه شوند. برای مثال، برای این که تضمین کنیم تمامی کاربران سرویس خوبی می‌گیرند، ممکن است بخواهیم حداکثر زمان پاسخ را کمینه کنیم.

تحلیل‌گران پیشنهاد می‌دهند که در سیستم‌های محاوره‌ای (همچون یک PC رومیزی یا یک سیستم لپ‌تاپ)، مهم‌تر است که به جای این که میانگین زمان پاسخ را کمینه کنیم، واریانس زمان پاسخ را به حداقل برسانیم. سیستمی که از یک زمان پاسخ مقبول و قابل پیش‌بینی برخوردار است، می‌تواند مطلوب‌تر از سیستمی باشد که در حالت متوسط سریع‌تر عمل می‌کند، اما تغییرات زیادی دارد. با این وجود، در الگوریتم‌های زمانبندی پردازنده که واریانس را کمینه می‌سازند کار کمتری انجام شده است.

همچنان که الگوریتم‌های زمانبندی مختلف پردازنده را در بخش بعدی بحث می‌کنیم عملیات آنها را نیز نشان خواهیم داد. در یک نمایش واقعی، بایستی پردازه‌های زیادی لحاظ شوند که هر یک دنباله‌ای از صدها انفجار پردازنده و انفجار I/O را دارند. هرچند که برای سادگی مثال‌های خود، تنها یک انفجار پردازنده از پردازه‌ها را (به میلی‌ثانیه) مد نظر قرار خواهیم داد. معیار مقایسه‌ی ما، میانگین زمان انتظار است. راهکارهای ارزیابی دقیق‌تر در بخش ۵-۸ مورد بحث قرار می‌گیرند.

۵-۳ الگوریتم‌های زمانبندی

زمانبندی پردازنده با مسأله‌ی تصمیم‌گیری این که، هسته‌ی پردازنده را در اختیار کدام یک از پردازه‌های صف آماده قرار بدهد، سر و کار دارد. الگوریتم‌های زمانبندی متفاوت پردازنده فراوان هستند. در این بخش، چند مورد از آنها را بیان می‌کنیم. هرچند که اکثر معماری‌های مدرن پردازنده دارای هسته‌های پردازشی متعددی هستند، ما این الگوریتم‌های زمان‌بندی را در متن فقط یک هسته‌ی پردازشی در دسترس بیان می‌کنیم. یعنی، یک پردازنده‌ی منفردی که دارای یک هسته‌ی پردازشی منفرد است، لذا سیستم، قابلیت اجرای یک پردازه را در یک لحظه دارا می‌باشد.

۵-۳-۱ زمانبندی اولین-ورود، اولین-سرویس

تاکنون، ساده‌ترین الگوریتم زمانبندی پردازنده، الگوریتم زمانبندی **اولین-ورود، اولین-سرویس**[1] (**FCFS**) بوده است. در این طرح، پردازه‌ای پردازنده را برای نوبت اول به‌دست می‌گیرد که اول از همه درخواست داده باشد. پیاده‌سازی سیاست FCFS به راحتی با یک صف FIFO انجام

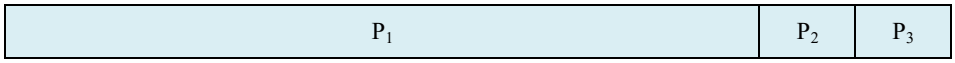
1. First-come, first-served scheduling algorithm

می‌شود. موقعی که پردازه‌ای به صف آماده وارد می‌شود، PCBی آن به ته صف می‌پیوندد. زمانی که پردازنده رها شد، در اختیار پردازه‌ای قرار می‌گیرد که سر صف قرار دارد. سپس، پردازه‌ی در حال اجرا از صف حذف می‌شود. کد زمانبندی FCFS به سادگی نوشته شده و فهم آن آسان است.

اما وجه منفی امر این است که میانگین زمان انتظار با سیاست FCFS بسیار طولانی است. مجموعه‌ی پردازه‌های زیر را که طول انفجار پردازنده‌های آنها بر حسب میلی‌ثانیه داده شده و در لحظه‌ی ۰ می‌رسند، در نظر بگیرید:

پردازه	زمان انفجار
P_1	۲۴
P_2	۳
P_3	۳

اگر پردازه‌ها با ترتیب P_1، P_2 و P_3 برسند و به ترتیب FCFS سرویس بگیرند، نتیجه‌ای به صورت **نمودار گانت**[1]، نمودار میله‌ای که یک زمانبندی مشخص را شامل زمان‌های شروع و پایان هر یک از پردازه‌های شرکت‌کننده نشان می‌دهد، نشان داده شده در زیر به‌دست می‌آوریم:



زمان انتظار پردازه‌ی P_1، ۰، P_2، ۲۴ و P_3، ۲۷ میلی‌ثانیه است. بنابراین، میانگین زمان انتظار (0 + 24 + 27)/3 = 17 میلی‌ثانیه می‌باشد. اما، اگر پردازه‌ها با ترتیب P_2، P_3 و P_1 می‌رسیدند، نتیجه به صورت نمودار گانت زیر می‌شد:



میانگین زمان انتظار فعلی (6 + 0 + 3)/3 = 3 میلی‌ثانیه است. این کاهش قابل توجه است. بنابراین، در حالت کلی، میانگین زمان انتظار با سیاست FCFS حداقل نبوده و می‌تواند با تغییرات زیاد ناشی از زمان‌های انفجار پردازنده‌ی پردازه‌ها به طور چشم‌گیری تغییر کند.

به علاوه، کارایی زمانبندی FCFS را در شرایط پویا در نظر بگیرید. فرض کنید یک پردازه پاپیچ پردازنده و پردازه‌های متعددی پاپیچ I/O داریم. همچنان که پردازه‌ها در سیستم جریان دارند، این وضع می‌تواند پیش بیاید. پردازه‌ی پاپیچ پردازنده، پردازنده را به‌دست آورده و نگه می‌دارد. در این میان، سایر پردازه‌ها همگی I/O خود را تکمیل کرده و به صف آماده می‌رسند که منتظر پردازنده بمانند. در حالی‌که پردازه‌ها در صف آماده منتظرند، دستگاه‌های I/O بیکار می‌مانند. بالاخره، پردازه‌ی پاپیچ پردازنده، انفجار پردازنده‌ی خود را تمام کرده و به یک دستگاه I/O جابه‌جا می‌شود. تمامی پردازه‌های پاپیچ I/O، که انفجارهای پردازنده‌ی بسیار کوتاهی دارند،

1. Gantt chart

به سرعت اجرا شده و دوباره به صف‌های I/O بر می‌گردند. اینجاست که پردازنده بیکار می‌نشیند. در ادامه، پردازه‌ی پاپیچ پردازنده، به صف آماده برگشته و پردازنده را در اختیار می‌گیرد. تمامی پردازه‌های پاپیچ I/O، دومرتبه کار خود را تمام کرده و در صف آماده به انتظار انجام پردازه‌ی پاپیچ پردازنده می‌مانند. در واقع یک **اثر بدرقه**[1] روی می‌دهد، طوری که سایر پردازه‌ها همگی منتظر یک پردازه‌ی بزرگ می‌مانند تا پردازنده را از آن بگیرند. پیامد این اثر آن است که بهره‌وری پردازنده و دستگاه‌های I/O پایین‌تر می‌آید، در حالی‌که اگر اجازه می‌دادیم پردازه‌های کوتاه‌تر در نوبت اول سرویس بگیرند، چنین وضعی پیش نمی‌آمد.

همچنین توجه کنید که الگوریتم زمانبندی FCFS پس‌گرفتنی نیست. همین که پردازنده به پردازه‌ای تخصیص می‌یابد، پردازه آن را نگه می‌دارد تا این که آن را خود، یا با خاتمه و یا با درخواست I/O رها کند. الگوریتم FCFS به ویژه در سیستم‌های محاوره‌ای مسأله‌دار است که در آن، هر پردازه در فواصل زمانی منظمی لازم دارد که از پردازنده سهمی ببرد. قابل تحمل نیست که پردازه‌ای برای دوره‌ی زمانی طولانی‌تری پردازنده را در اختیار بگیرد.

۵-۳-۲ زمانبندی کوتاهترین-کار-اول

رویکرد دیگر متفاوت زمانبندی پردازنده، الگوریتم زمانبندی **کوتاهترین-کار-اول**[2] (**SJF**) است. این الگوریتم به هر یک از پردازه‌ها، طول انفجار بعدی پردازنده‌اش را نسبت می‌دهد. زمانی که پردازنده در دسترس باشد به پردازه‌ای گمارده می‌شود که کوچکترین انفجار بعدی پردازنده را دارد. اگر انفجارهای بعدی پردازنده‌ی دو پردازه یکسان باشند، برای شکستن گره میان آنها از زمانبندی FCFS استفاده می‌شود. توجه داشته باشید که اصطلاح مناسب این روش، زمانبندی *الگوریتم کوتاه‌ترین-انفجار-بعدی-پردازنده*[3] خواهد بود، چون زمانبندی آن به جای طول کل پردازه به طول انفجار بعدی پردازنده‌ی یک پردازه بستگی دارد. چون بیشتر افراد و کتاب‌ها برای رجوع به این نوع زمانبندی از تعبیر SJF استفاده می‌کنند، ما نیز از اصطلاح SJF استفاده می‌کنیم.

به عنوان مثالی از زمانبندی SJF، مجموعه‌ی پردازه‌های زیر را که طول زمان انفجار پردازنده‌ی آنها بر حسب میلی‌ثانیه به صورت زیر داده شده‌اند در نظر بگیرید:

پردازه	زمان انفجار
P_1	۶
P_2	۸
P_3	۷
P_4	۳

این پردازه‌ها را با استفاده از زمانبندی SJF، مطابق نمودار گانت زیر زمانبندی خواهیم کرد:

1. Convoy effect
2. Shortest-job-first algorithm
3. Shortest-next-CPU-burst algorithm

P_4	P_1	P_3	P_2
۰ ۳	۹	۱۶	۲۴

زمان انتظار پردازه‌ی P_1، ۳، P_2، ۱۶، P_3، ۹ و P_4، ۰ میلی‌ثانیه می‌باشد. بنابراین، میانگین زمان انتظار $(3 + 16 + 9 + 0)/4 = 7$ میلی‌ثانیه است. در مقایسه، اگر از طرح زمانبندی FCFS استفاده می‌کردیم، میانگین زمان انتظار ۱۰/۲۵ میلی‌ثانیه می‌شد.

قابل اثبات است که الگوریتم زمانبندی SJF، بهینه می‌باشد، یعنی در آن، برای مجموعه‌ای از پردازه‌ها، کمترین میانگین زمان انتظار به‌دست می‌آید. جابه‌جایی یک پردازه با انفجار کوتاه به قبل از یک پردازه با انفجار طولانی، زمان انتظار پردازه‌ی کوتاه را بیش از افزایش زمان انتظار پردازه‌ی طولانی کاهش می‌دهد. در نتیجه، میانگین زمان انتظار کاهش می‌یابد.

اگرچه الگوریتم زمانبندی SJF بهینه است، نمی‌تواند در سطح زمانبندی پردازنده پیاده‌سازی شود، چرا که هیچ راهی برای دانستن طول انفجار بعدی پردازنده وجود ندارد. یک رویکرد برای مسأله این است که سعی کنیم زمانبندی SJF را تقریب بزنیم. ممکن است که طول انفجار بعدی پردازنده را ندانیم، اما این امکان وجود دارد که مقدار آن را پیش‌بینی کنیم. انتظار داریم که انفجار بعدی پردازنده با طول انفجارهای قبلی، مشابهت زیادی خواهد داشت. بنابراین، با محاسبه‌ی طول تقریبی انفجار بعدی پردازنده می‌توانیم پردازه‌ای را انتخاب کنیم که کوتاه‌ترین انفجار پیش‌بینی پردازنده را خواهد داشت.

به طور کلی، انفجار بعدی پردازنده به صورت **میانگین نمایی**[1] از طول انفجارهای اندازه‌گیری قبلی پردازنده پیش‌بینی می‌شود. می‌توانیم میانگین نمایی را با فرمول زیر تعریف کنیم. اگر t_n، طول nامین انفجار پردازنده و τ_{n+1}، پیش‌بینی ما از مقدار بعدی انفجار پردازنده باشد. در این صورت، به ازای هر α، که $0 \le \alpha \le 1$، می‌توان فرمول زیر را تعریف کرد:

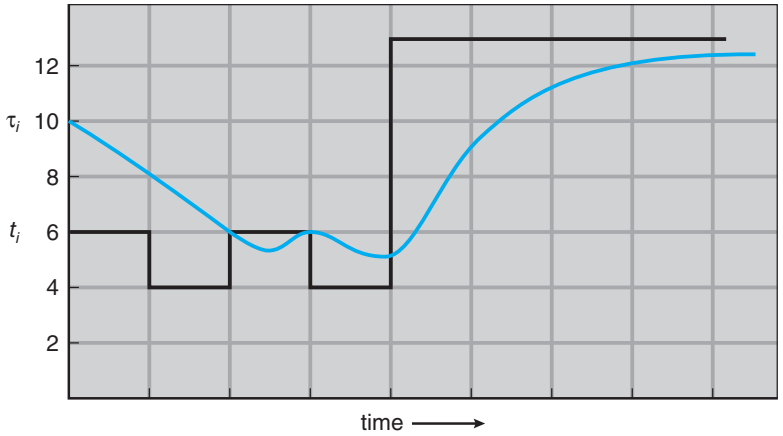
$$\tau_{n+1} = \alpha t_n + (1-\alpha)\tau_n$$

مقدار t_n، اخیرترین اطلاع ما را نشان می‌دهد، در حالی‌که τ_n تاریخ گذشته را ذخیره می‌کند، پارامتر α، وزن نسبی اخیر و تاریخ گذشته را در پیش‌بینی ما کنترل می‌کند. اگر $\alpha = 0$ باشد، در این صورت $\tau_{n+1} = \tau_n$ بوده و تاریخ اخیر بی‌تأثیر است (یعنی، شرایط فعلی گذرا فرض می‌شوند)؛ اگر $\alpha = 1$ باشد، در این صورت $\tau_{n+1} = t_n$ خواهد بود و فقط اخیرترین انفجار پردازنده اهمیت می‌یابد (به نظر می‌رسد که تاریخ، قدیمی شده و نامربوط است). با انتخاب $\alpha = 1/2$ که بیشتر مرسوم است، تاریخ اخیر و تاریخ گذشته از وزن یکسانی برخوردار می‌شوند. می‌توان τ_0 اولیه را به صورت یک ثابت یا به عنوان میانگین کلی سیستم تعریف کرد. شکل ۵-۴، میانگین نمایی را با $\alpha = 1/2$ و $\tau_0 = 10$ نشان می‌دهد.

1. Exponential average

برای فهم رفتار میانگین نمایی می‌توانیم فرمول τ_{n+1} را با جایگزینی τ_n بسط داده و رابطه‌ی زیر را به‌دست آوریم:

$$\tau_{n+1} = \alpha t_n + (1-\alpha)\alpha t_{n-1} + \cdots + (1-\alpha)^j \alpha t_{n-j} + \cdots + (1-\alpha)^{n+1} \tau_0$$



شکل ۵-۴ پیش‌بینی طول انفجار بعدی پردازنده.

نوعاً، α از ۱ کمتر است. در نتیجه، $(1-\alpha)$ نیز از ۱ کمتر بوده و وزن هر یک از جملات بعدی نسبت به جمله‌ی قبلی کمتر می‌شود.

الگوریتم SJF می‌تواند *پس‌گرفتنی* و یا *غیرقابل پس‌گیر* باشد. انتخاب موقعی است که پردازه‌ی جدیدی به صف آماده می‌رسد و در همان حال، پردازه‌ای از قبل همچنان در حال اجراست. ممکن است انفجار بعدی پردازنده‌ی پردازه‌ی جدید، کوتاه‌تر از میزان باقیمانده‌ی انفجار پردازه‌ی فعلی در حال اجرا باشد. الگوریتم SJF پس‌گرفتنی، اجرای پردازه‌ی فعلی را متوقف می‌کند، در حالی‌که الگوریتم SJT غیرقابل پس‌گیر اجازه می‌دهد تا پردازه‌ی در حال اجرای فعلی، انفجار پردازنده‌ی خود را به اتمام برساند. گاهی زمانبندی SJF پس‌گرفتنی، زمانبندی **کوتاه‌ترین زمان باقیمانده اول**[1] نامیده می‌شود.

به عنوان یک مثال، چهار پردازه‌ی زیر را که طول زمان انفجار پردازنده‌ی آنها برحسب میلی‌ثانیه داده شده است، در نظر بگیرید:

پردازه	زمان ورود	زمان انفجار
P_1	۰	۸
P_2	۱	۴
P_3	۲	۹
P_4	۳	۵

1. Shortest-remaining-time-first

اگر ورود پردازه‌ها با زمان‌های نشان داده شده و زمان انفجار مورد نیاز آنها نیز مقادیر نشان داده شده باشند، در این صورت زمانبندی SJF پس‌گرفتنی آنها، نمودار گانت زیر را خواهد داشت:

P_1	P_2	P_4	P_1	P_3
۰ ۱		۵	۱۰	۱۷ ۲۶

پردازه‌ی P_1 اجرای خود را در زمان ۰ آغاز می‌کند، چون تنها پردازه‌ای است که در صف قرار دارد. پردازه‌ی P_2 در زمان ۱ می‌رسد. باقیمانده‌ی زمان انفجار پردازه‌ی P_1 (۷ میلی‌ثانیه) از زمان انفجار مورد نیاز پردازه‌ی P_2 (۴ میلی‌ثانیه) بیشتر است، لذا پردازه‌ی P_1 کنار گذاشته شده و پردازه‌ی P_2 زمانبندی می‌شود. میانگین زمان انتظار این مثال برابر [(10 -1) + (1 – 1) + (17 – 2) + (5 – 3)]/4 = 26/4 = 6.5 میلی‌ثانیه خواهد بود. میانگین زمان انتظار زمانبندی SJF غیرقابل پس‌گیر نیز برابر ۷/۷۵ میلی‌ثانیه می‌شود.

۵-۳-۳ زمانبندی نوبت-چرخشی

الگوریتم زمانبندی **نوبت-چرخشی**[1] (**RR**)، شبیه زمانبندی FCFS است، اما در جابه‌جایی میان پردازه‌ها پس‌گیری هم اضافه می‌شود. واحد زمانی کوچکی به نام **کمیت زمانی**[2] یا **برش زمانی** تعریف می‌شود. طول کمیت زمانی به طور کلی بین ۱۰ تا ۱۰۰ میلی‌ثانیه است. صف آماده همچون یک صف چرخشی در نظر گرفته می‌شود. زمانبند پردازنده در صف آماده دور می‌زند و پردازنده را حداکثر تا ۱ کمیت زمانی در اختیار هر یک از پردازه‌ها قرار می‌دهد.

برای پیاده‌سازی زمانبندی RR، باز هم با صف آماده به صورت یک صف FIFO رفتار می‌کنیم. پردازه‌های جدید به ته صف آماده اضافه می‌شوند. زمانبند پردازنده، اولین پردازه را از صف آماده برداشته و یک تایمر تنظیم می‌کند تا بعد از یک کمیت زمانی وقفه بدهد و پردازه را اعزام می‌کند.

در ادامه، یکی از این دو حالت اتفاق می‌افتد. انفجار پردازنده‌ای که پردازه داشته، کمتر از ۱ کمیت زمانی است. در این حالت، پردازه به صورت داوطلبانه پردازنده را خود رها می‌کند. زمانبند با پردازه‌ی بعدی صف آماده کار را پیش می‌برد. اگر انفجار پردازنده‌ی فعلی پردازه‌ی در حال اجرا بیشتر از ۱ کمیت زمانی باشد، تایمر به سر آمده و وقفه‌ای برای سیستم‌عامل صادر خواهد کرد. یک برگردان متن صورت می‌گیرد و پردازه دومرتبه در ته صف آماده قرار خواهد گرفت. سپس زمانبند پردازنده، پردازه‌ی بعدی صف آماده را انتخاب خواهد کرد.

میانگین زمان انتظار با سیاست RR اغلب طولانی است. مجموعه‌ی پردازه‌های زیر را در نظر بگیرید که در لحظه‌ی ۰ با طول زمان انفجار پردازنده‌ی داده شده برحسب میلی‌ثانیه می‌رسند:

1. Round-Robin
2. Time quantum

پردازه	زمان انفجار
P_1	۲۴
P_2	۳
P_3	۳

اگر از کمیت زمانی ۴ میلی‌ثانیه‌ای استفاده کنیم، در این صورت پردازه‌ی P_1، اولین ۴ میلی‌ثانیه را بدست می‌آورد. چون ۲۰ میلی‌ثانیه‌ی دیگر هم لازم دارد، بعد از اتمام اولین کمیت زمانیش، پردازنده از آن پس‌گرفته شده و در اختیار پردازه‌ی بعدی صف، P_2 قرار می‌گیرد. چون پردازه‌ی P_2 کمتر از ۴ میلی‌ثانیه نیاز دارد، لذا پیش از آنکه کمیت زمانیش تمام شود از صف خارج می‌شود. سپس پردازنده به پردازه‌ی بعدی، یعنی P_3 داده می‌شود. بعد از این که هر یک از پردازه‌ها، ۱ کمیت زمانی دریافت کردند، پردازنده دومرتبه با کمیت زمانی دیگری در اختیار P_1 قرار می‌گیرد. زمانبندی RR حاصل به صورت زیر خواهد بود:

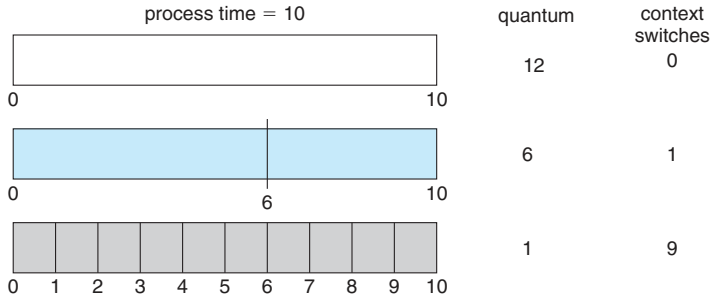
P_1	P_2	P_3	P_1	P_1	P_1	P_1	P_1
۰ ۴	۷	۱۰	۱۴	۱۸	۲۲	۲۶	۳۰

بیایید میانگین زمان انتظار را برای زمانبندی بالا حساب کنیم. پردازه‌ی P_1 به اندازه‌ی 6 = (4 - 10) میلی‌ثانیه، P_2 به اندازه‌ی ۴ میلی‌ثانیه و P_3 برابر ۷ میلی‌ثانیه منتظر خواهند ماند. لذا، میانگین زمان انتظار برابر 17/3 = 5.66 میلی‌ثانیه می‌شود.

در الگوریتم زمانبندی RR هیچ پردازه‌ای نمی‌تواند پردازنده را در یک دور بیش از ۱ کمیت زمانی در اختیار بگیرد (مگر این که تنها پردازه‌ی در حال اجرا باشد). اگر انفجار پردازنده‌ی پردازه‌ای از ۱ کمیت زمانی فراتر رود، پردازه *پس‌گرفته* شده و دومرتبه در صف آماده قرار داده می‌شود. بنابراین، الگوریتم زمانبندی RR پس‌گرفتنی است.

اگر تعداد n پردازه در صف آماده داشته باشیم و کمیت زمانی q باشد، هر یک از پردازه‌ها، $1/n$ زمان پردازنده را با قالب‌هایی که حداکثر آن q واحد زمانی است در اختیار خواهند گرفت. هر یک از پردازه‌ها مجبور نخواهند بود که برای رسیدن نوبت بعدی کمیت زمانی خود، بیش از $(n - 1) \times q$ واحد زمانی انتظار بکشند. برای مثال، با پنج پردازه و کمیت زمانی ۲۰ میلی‌ثانیه‌ای، هر یک از پردازه‌ها در هر ۱۰۰ میلی‌ثانیه، پردازنده را حداقل ۲۰ میلی‌ثانیه در اختیار می‌گیرند.

کارایی الگوریتم RR، وابستگی زیادی به اندازه‌ی کمیت زمانی دارد. در حد نهایت، اگر کمیت زمانی را خیلی بزرگ بگیریم، سیاست RR، همانند سیاست FCFS خواهد بود. در مقایسه، اگر کمیت زمانی بی‌نهایت کوچک باشد (برفرض، ۱ میلی‌ثانیه)، رویکرد RR می‌تواند به تعداد زیادی برگردان متن منجر شود. برای مثال، فرض کنید فقط یک پردازه با ۱۰ واحد زمانی داریم. اگر کمیت برابر ۱۲ واحد زمانی باشد، پردازه در کمتر از ۱ کمیت زمانی، بدون هیچ‌گونه سرباری به اتمام می‌رسد. با این همه، اگر کمیت برابر ۶ واحد زمانی باشد، پردازه نیازمند ۲ کمیت است که یک برگردان متن در پی دارد. اگر کمیت زمانی برابر ۱ واحد زمانی باشد، نه برگردان متن اتفاق افتاده و متعاقباً اجرای پردازه کندتر می‌شود (شکل ۵-۵).



شکل ۵-۵ چگونگی افزایش تعداد برگردان متن با کوچک‌ترشدن کمیت زمانی.

بنابراین، می‌خواهیم کمیت زمانی در مقایسه با زمان برگردان متن تا حدودی بزرگ باشد. اگر زمان برگردان متن، معادل ۱۰ درصد کمیت زمانی باشد، در این صورت حدود ۱۰ درصد زمان پردازنده صرف برگردان متن خواهد شد. کمیت زمانی بیشتر سیستم‌های مدرن، در عمل در بازه‌ی ۱۰ تا ۱۰۰ میلی‌ثانیه تغییر می‌کند. زمان مورد نیاز نوعی برگردان متن کمتر از ۱۰ میکروثانیه می‌باشد؛ بنابراین، زمان برگردان متن فقط کسر کوچکی از کمیت زمانی است.

زمان گردش نیز به اندازه‌ی کمیت زمانی بستگی دارد. همان‌گونه که می‌توانیم از شکل ۵-۶ ببینیم، میانگین زمان گردش مجموعه‌ای از پردازه‌ها، لزوماً با افزایش اندازه‌ی کمیت زمانی بهبود پیدا نمی‌کند. به طور کلی، موقعی در میانگین زمان گردش بهبودی حاصل می‌شود که بیشتر پردازه‌ها انفجار بعدی پردازنده‌ی خود را تنها با یک کمیت زمانی منفرد به اتمام برسانند. برای مثال، اگر سه پردازه که انفجار هر یک از آنها ۱۰ واحد زمان است با کمیت زمانی ۱ واحد داشته باشیم، در این صورت میانگین زمان گردش برابر ۲۹ خواهد بود. با این وجود، اگر کمیت زمانی ۱۰ باشد، میانگین زمان گردش به ۲۰ کاهش می‌یابد. در صورتی‌که زمان برگردان متن نیز به حساب بیاید، چون با کمیت زمانی کوچکتر، برگردان متن بیشتری لازم خواهد شد، میانگین زمان گردش حتی افزایش بیشتری هم خواهد داشت.

اگرچه کمیت زمانی می‌بایست در مقایسه با زمان برگردان متن تا حدی بزرگ انتخاب شود، نباید زیادی هم بزرگ باشد. همان‌گونه که پیشتر نشان دادیم، اگر کمیت زمانی خیلی بزرگ باشد، زمانبندی RR به یک سیاست FCFS تغییر شکل می‌دهد. یک قاعده‌ی سرانگشتی این است که بایستی ۸۰ درصد انفجارهای پردازنده کوتاه‌تر از کمیت زمانی باشند.

۵-۳-۴ زمانبندی الویتی

الگوریتم SJF، حالت خاصی از حالت کلی الگوریتم **زمانبندی الویتی** محسوب می‌شود. برای هر یک از پردازه‌ها، الویتی نسبت داده می‌شود و پردازنده در اختیار پردازه‌ای قرار می‌گیرد که بالاترین الویت را دارد. پردازه‌های با الویت یکسان، به ترتیب FCFS زمانبندی می‌شوند. هر الگوریتم SJF، یک الگوریتم الویتی ساده می‌باشد که الویت (p) آن، عکس انفجار بعدی پردازنده (پیش‌بینی‌شده) است. اگر انفجار پردازنده بزرگ‌تر باشد، الویت آن کمتر است و برعکس.